FleetPulse: how it works, and what it still needs

Every Vehicle. Every Journey. Under Control.

Part 1 explains the project in plain language. Part 2 is a spoken walkthrough you can follow to present or demonstrate it. Part 3 lists what only you can provide. Part 4 lists what's still missing from the product itself.

Where the project stands

All 23 planned phases (0 to 22) are built and tested on one development machine.

FleetPulse has never run on a real server, with real customers, a real GPS device or a real payment. Those steps need accounts and decisions that only you can provide, so "built and tested" isn't yet "live".

Check, in the final run	Result
Backend tests, unit and integration	834 passed (109 + 725)
Backend code covered by the integration tests (last measured in Phase 21)	95%
Frontend unit tests	336 passed
Browser tests against the real system (Chromium, phone, Firefox)	415 passed, 3 skipped by design
Browser tests in development mode	60 passed
Production setup, checked locally with HTTPS	4 of 4 journeys passed
Backup, then restore into a new database	Restored data matched
Known vulnerabilities in dependencies	0

Nothing is saved in git yet. All of this work exists only on this one computer. Commit it and push it to a private repository before anything else, because a disk failure would lose the whole project. CI has also never run, because it needs that first push.

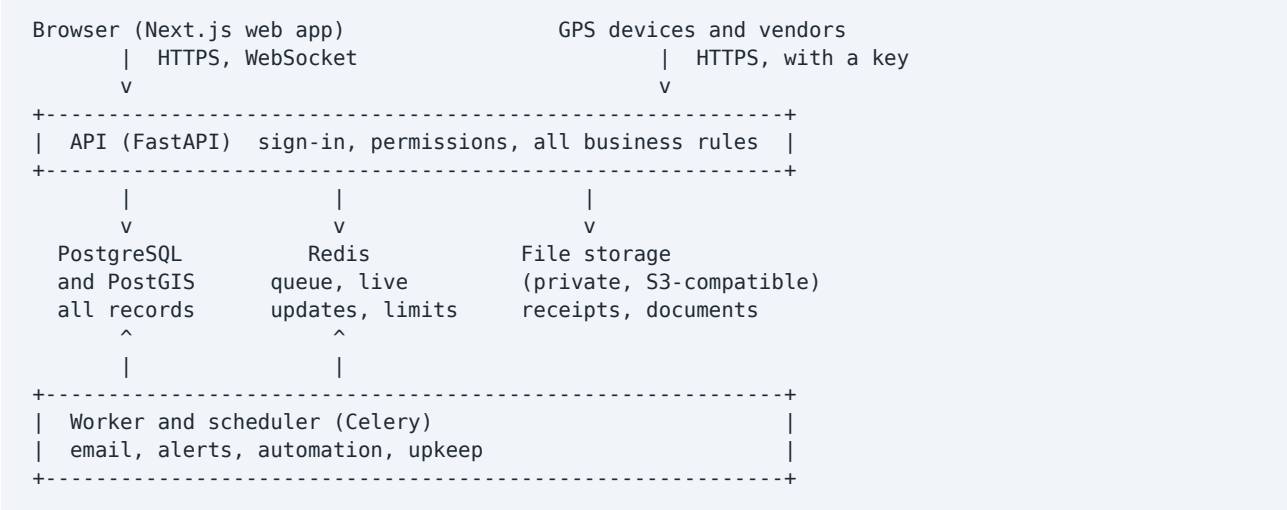
Part 1: How the project works

What it is

FleetPulse is software that organisations subscribe to for running their vehicles. A logistics company, a school with buses or a delivery business signs up and gets one place to see where its vehicles are, plan trips, keep vehicles maintained and legal, and control what the fleet costs.

It's multi-tenant: many organisations share one system, and none can see another's data.

The main parts



- **Web app:** what people see. It holds no business rules, and the API checks everything again.
- **API:** the only way to read or change data.
- **Database:** PostgreSQL stores the records. PostGIS, its location extension, answers questions such as "is this position inside this area?"
- **Redis:** carries live map updates, background jobs and rate-limit counters.
- **File storage:** receipts and documents, kept private and reached through links that expire in 5 minutes.
- **Worker and scheduler:** do the work nobody is waiting on, such as sending email, finding vehicles that have gone silent, and running automation rules.

What happens when someone uses it

Signing up. A person registers and confirms their email. Passwords are stored only as Argon2id hashes, so even the database doesn't hold the password itself. Sign-in uses a 15-minute token in a cookie that browser scripts can't read, renewed by a second token that's replaced each time it's used. If a used token ever reappears, the session is ended, because that suggests theft. Google sign-in is built, and stays hidden until you add credentials.

Creating an organisation. The person creates an organisation and invites colleagues, each with a role:

Role	In short
Owner, Administrator	Everything
Fleet manager	Runs the fleet day to day, and sets up automation
Dispatcher	Plans and runs trips

Role	In short
Maintenance manager	Work orders, schedules and vehicle paperwork
Finance manager	Approves expenses
Driver	Sees only their own trips, vehicle, records and documents
Read-only	Looks, and changes nothing

Keeping organisations apart. This is enforced twice. The API always takes the organisation from the signed-in session, never from the request. Then the database applies row-level security, so even a bug in the API can't return another organisation's rows.

A position arrives. A GPS device or vendor sends a position using a secret key:

1. The position is checked and stored.
2. In the same database transaction, FleetPulse checks whether the vehicle crossed a geofence, is speeding, or is moving with its ignition off.
3. Anyone watching the live map gets the update through a WebSocket, within about a second. If that connection drops, the page says so and refreshes every 30 seconds.

A problem is noticed. Alerts come from positions (geofences, speed, unusual movement), and from a check that runs every minute (silent vehicles, expiring documents, maintenance due). Every 30 seconds a job picks up new alerts, notifies the right people in the app, emails them if their preferences say so, and runs matching automation rules. A failed email retries up to 5 times, and the screen says if it finally failed.

A trip runs. Trips move through draft, planned, dispatched, in progress and completed, and can be cancelled before completion. FleetPulse refuses to dispatch a vehicle in maintenance, a vehicle with a critical defect, a suspended driver or an expired licence. It checks again at departure, because any of those can change in between. When a trip finishes, distance is measured from GPS positions and labelled as GPS, simulated or typed in.

Keeping vehicles legal and running.

- **Maintenance:** schedules come due by date or odometer, whichever is first. Starting a work order takes the vehicle off the road, and it returns when the work ends.
- **Inspections:** no check can default to a pass. A failed critical check creates a critical defect, which stops the vehicle being dispatched until it's fixed.
- **Documents:** insurance, roadworthiness and licences get reminders 30 days before they expire.

Money. Fuel and expenses can be recorded in any currency, and amounts in different currencies are never added together or converted. Someone other than the person who submitted an expense has to approve it, unless that person is an owner. Cost per kilometre appears only when two odometer readings make it measurable. Financial records are voided, not deleted.

Reports. Nine reports with date filters and CSV and PDF exports: utilisation, trips, drivers, fuel, expenses, maintenance, fleet performance, compliance and alerts.

Billing. Prices are per vehicle for each 30 days, with a 14-day trial. A payment counts only when the payment provider's signed message confirms it, with the amount checked. Billing is off until you switch it on.

Staff administration. FleetPulse staff use a hidden area at `/admin`. It needs a flag set from the server plus a password check, shows account-level facts only, and offers no way to read a customer's routes or sign in as them. Suspending an organisation or deactivating a user needs a written reason, which goes in the audit log.

The rule behind the design: never pretend

The product is built not to show anything it can't back up:

- An unknown speed shows as unknown, never 0.
- A vehicle's position turns stale, then offline, as it ages, even while the page stays open.
- Simulated data is always labelled.
- A document with no expiry date isn't called valid.
- A vehicle with no insurance on file isn't reported as uninsured.
- Reports say what they leave out.

How it's protected

- Passwords are hashed with Argon2id, and login attempts are rate limited.
- Cross-site attacks are blocked by checking the request's origin, a CSRF token and the cookie rules.
- A strict Content-Security-Policy stops script injected into a page from running. This was tested in Chromium and Firefox.
- Uploads are identified from the file's own bytes, limited to 10 MB, and always download rather than open in the browser.
- Important actions go in an audit log that the app's database account can't change or delete.
- Production refuses to start with an unsafe setup, such as no STARTTLS on email or the GPS simulator switched on.

Technology

Area	Used
Web app	Next.js 16, React 19, TypeScript, Tailwind, DaisyUI, TanStack Query, MapLibre
API	Python 3.13, FastAPI, SQLModel, Alembic
Data	PostgreSQL 17 with PostGIS, Redis, S3-compatible storage
Background work	Celery
Testing	pytest, Vitest, Playwright, axe (accessibility)
Deployment	Docker Compose, Caddy (automatic HTTPS)

Running it on your computer

Start the backend services:

```
docker compose --profile app up -d --build --wait
```

In another terminal, start the website:

```
cd frontend && npm run dev
```

To see	Go to
The site	http://localhost:3210
Emails it sends, including sign-up links	http://localhost:8027

To make your own account staff, once its email is verified, run this from `backend/`:

```
uv run python -m app.db.make_platform_admin you@example.com
```

Where to find things

Looking for	Go to
Setup and commands	`README.md`
Deploying, backups and fixing problems	`docs/operations.md`
Why each big decision was made	`docs/adr/`
Connecting a GPS device or vendor	`docs/gps-ingestion.md`
The original requirements	`docs/product-spec.md`

Part 2: A spoken walkthrough

Use this to present or demonstrate the project. It takes about 10 minutes. Beforehand, make sure `FLEETPULSE_GPS_SIMULATOR_ENABLED=true` is set in `.env`, then restart the containers, and create one simulated device so the map moves.

Opening. "Running vehicles means juggling locations, drivers, repairs, fuel and paperwork, often across separate systems. FleetPulse brings them together. Many organisations share the system, and each one's data stays completely separate."

1. The website. Open the home page. "The map here is a demonstration, and it says so. Honesty runs through this whole product: it never shows data it can't back up."

2. Sign up. Register, then open the email inbox and verify. "Passwords are never stored, only a strong hash, and the sign-in token can't be read by page scripts."

3. Set up the fleet. Create an organisation, add a vehicle and add a driver. "Currency, time zone and distance units belong to each organisation, so no single country is assumed."

4. The live map. Open Tracking, add a simulated device, and open Live map. "Positions arrive over a WebSocket. Simulated data is always labelled. If a vehicle stops reporting, its age keeps counting, and it turns stale and then offline by itself."

5. A geofence. Draw one around the vehicle's area and mark it restricted. "It uses PostGIS. An entry raises a critical alert, in the same transaction as the position, so a crash can't lose it. Drawing a fence around a parked vehicle doesn't raise a false alarm."

6. Safety in action. This is the strongest moment.

1. Create a trip for the vehicle and driver, and mark it planned.
2. Create an inspection template with a critical check, run the inspection, and fail that check.
3. Try to dispatch the trip. It's refused.
4. Say: "A failed safety check stops the vehicle, and that's enforced by the system rather than left to memory."
5. Create a work order from the defect, start it and complete it. Dispatch now works.

7. Costs. Add a fuel record with a receipt, then an expense, and approve it. "Someone else has to approve an expense, and currencies are never mixed. Cost per kilometre only appears when it can actually be measured."

8. Automation and notifications. Show a rule such as "when a critical defect is reported, create a work order", and its history. "Rules come from a fixed list, so no user-written code ever runs. Every run is recorded, and failures retry and then show as failed."

9. Reports. Open a report and download the PDF. "Each report states its own limits. Blank means unknown, never zero."

10. Security. Open `/admin` from a normal account, and it looks like a missing page. "Staff access needs a flag set on the server plus a password, and even staff can't read a customer's routes. Beneath the API, the database also keeps organisations apart."

Close, and be upfront. "It has over 1,500 automated tests, including real browser journeys. It's been checked with HTTPS on one machine, but hasn't yet run on a real server, or with a real GPS device or payment. Those are the next steps."

Questions you may be asked

Question	Answer
Is the location data real?	The pipeline is real, and the positions in a demo are simulated and labelled. It's ready for a real device, and hasn't yet been tested with one.
Can one company see another's data?	No. It's blocked in the API and again in the database, with tests for each area.
Does it take real payments?	Not yet. The billing code is tested against a stand-in for Paystack, and needs a real account.
How many vehicles can it handle?	Tested with 2,000 vehicles, on one overloaded development machine. Production capacity hasn't been measured.
What happens when GPS drops out?	The vehicle turns stale and then offline, and an alert fires unless it was last seen parked.

Part 3: What you need to provide

No software is ever truly perfect. What will make FleetPulse complete and ready for customers is the list below. I can do the technical setup for each once you have them. Costs and providers change, so check current terms before signing up.

Do this first

#	What	Why
1	**Tell me to commit and push to a private repository**	All the work is on one computer. This also runs CI for the first time, and CI may need fixes because it's never run.

Needed to go live

#	What you provide	What it unlocks	Where it goes
2	**A domain name**, with two addresses such as `app.yourdomain.com` and `api.yourdomain.com`	A real address and HTTPS	`deploy/.env.production`
3	**A server**, with Docker and at least 4 GB of memory	Somewhere for it to run	Follow `docs/operations.md`
4	**An email provider** with SMTP, and a verified sending domain	Sign-up emails, password resets and alerts. Nobody can register without this.	`FLEETPULSE_SMTP_*`
5	**A private S3-compatible bucket and keys**, from any provider	Receipts and documents	`FLEETPULSE_STORAGE_*`
6	**Somewhere off the server to keep backups**	Surviving the loss of the server	`deploy/backup.sh`, scheduled
7	**A contact inbox**	Messages from the website's contact form	`FLEETPULSE_CONTACT_INBOX_EMAIL`

Needed to do its real job

#	What you provide	What it unlocks
8	**A GPS vendor or device**, ideally one you can test with. Tell me its name and share its documentation.	Real vehicle tracking, which is the heart of the product. Without it, only simulated data moves on the map. A vendor with its own data format needs a small adapter, which I can write.

Needed to charge money

#	What you provide	What it unlocks
9	**A payment provider account.** I built for Paystack, so tell me if you'd prefer another.	Paid subscriptions, starting with a test payment
10	**Confirmed prices.** The $9 and $15 per vehicle, the plan limits and the 14-day trial were my choices, so please confirm or change them.	The prices customers will actually see
11	**A registered business**, and advice on tax	Invoices and tax. These aren't built, because they depend on where you're registered.

Needed legally

#	What you provide	Why
12	**Your company's legal name, address, contact email and governing law**	They appear on the Terms and Privacy pages, which show gaps until they're set
13	**A lawyer's review of the Terms and Privacy pages.** I drafted them, but I'm not a lawyer.	Tracking vehicles can mean tracking people, so data protection law applies. Nigeria's NDPA and the EU's GDPR are examples, depending on where you operate.
14	**A decision, with that lawyer, on how long to keep data.** Location history is kept for 13 months by default. Alerts and events have no limit yet.	I can build whatever you decide

Optional, but worthwhile

#	What you provide	What it unlocks
15	**Google sign-in credentials**	The "Sign in with Google" button appears
16	**Your brand assets:** logo, colours and final wording	Your identity instead of my placeholder design
17	**An uptime monitor and error tracking account**, such as Sentry	Being told when something breaks, rather than hearing it from a customer
18	**An independent security test**	A fresh set of eyes. My review can't replace one, and business customers may ask for it.
19	**A decision on maps.** OpenFreeMap is free, with no uptime guarantee.	A paid map provider if reliability matters. It's a one-line setting.
20	**Real users to try it**	The most valuable item here. Real fleets will find what no test can.

If you only do a few things

1. Commit and push the code.
2. Get a domain, a server, email and storage, and go live for testing.
3. Connect one real GPS device.
4. Have a lawyer review the legal pages.
5. Put it in front of one real fleet.

Part 4: What's still missing from the product

These are things I could build, but they aren't done yet. Real users should decide which matter.

Area	Gap
Billing	No automatic renewal, so each 30-day period is paid by hand. No invoices or tax handling. Vehicles added mid-period aren't charged until the next payment.
GPS	Only a documented generic feed so far. Vendor-specific adapters come once you choose one.
Fuel	Litres and kWh only, with no gallons.
Currencies	Totals are never converted between currencies.
Trips	Origins and destinations are names only, so there's no route planning or estimated distance before departure. Distance adds straight lines between positions, so it can read short on winding roads.
Mobile	There is no mobile app, and a phone can't be used as a tracker. The web app does work on phones.
Geofences	Apply to every vehicle, not selected ones. Late-arriving positions don't record crossings.
Alerts	Text is stored in kilometres, even for organisations using miles. Emails have no one-click unsubscribe.
Maintenance	Odometers are typed in, not read from GPS. Provider details are per work order, with no shared directory. A finished work order can't be corrected.
Accounts	Deleting an account and exporting data aren't self-service, so people have to ask you. Sign-up shows a notice about the Terms, but doesn't record acceptance.
Reports	Tables and headline figures only, with no charts. Long periods over very large fleets could be slow.
Storage	Uploads never attached to a record aren't cleaned up.
Scale	One server to start with, so it's a single point of failure. Live updates across several servers are designed for but untested.
Testing	Frontend code coverage hasn't been measured, and production capacity hasn't been measured on real hardware.